AI FOR PRODUCT MANAGERS · IIIT HYDERABAD · CAPSTONE PROJECT · PRD

SpecBridge: Jira tickets to codebase-grounded UAT test cases

Team Unmanned Rooks: Hussain Lohawala, Qasim Naik, Rishabh Singh, Dhawal **Version** v0.1 · 28 Oct 2026

Today a PM sees the user journey only after engineers have changed 3-5 repositories, and if it is not what the PM meant, the work goes round again. SpecBridge reads the team's real code and shows the PM that journey first, as UAT test cases (steps, expected results, affected screens, open questions), so PM and engineers agree on it before anything is built.

1 User and job-to-be-done

USER

A product manager who hands UI tasks to frontend and backend engineers on a product whose frontend is split across several repositories.

JOB-TO-BE-DONE

When I hand over a UI task, I want to see and agree the exact user journey, including errors and edge cases, before engineers build it, so the first demo is what I meant.

Secondary users: the frontend engineer, who builds from an agreed journey instead of guessing, and QA, who later runs the same cases.

2 Problem

How it works today:

1. The PM gives a UI task to the frontend and backend engineers.
2. The frontend engineer translates it into changes across 3-5 repositories.
3. Only when the work is done does the engineer demo the user journey to the PM.
4. If it is not what the PM meant, the cycle repeats, sometimes several times.

Why: the journey is never made concrete and agreed before building. The PM writes in product language ("let users finalize a bill") and does not know which screens and flows across the repos the change touches, so mismatches, error states and edge cases surface at the demo, the most expensive moment to find them.

Why now: current LLMs can read code and plain language together, and a ticket costs cents to process.

3 Approach



- **Understand** the ticket: criteria (stated and inferred), screens, questions, risks. Unclear or unsafe tickets get questions, not invented cases.
- **Agents per acceptance criterion** search a code knowledge base (keywords + local embeddings + links between files across repos) with read-only tools, then write cases that cite the code they read.
- **An independent reviewer** checks coverage and that every cited screen exists; weak criteria go back for another round.
- **Model:** Gemini 3.8 Flash via OpenRouter; the model can be swapped through configuration. Built with LangGraph.

4 Example

THE TICKET, AS THE PM WRITES IT

"Start visit" does nothing from patient search

When the front-desk user searches for a patient and clicks **Start visit** on a result, nothing happens. Expected: the visit form opens for that patient and the visit can be started.

WHAT SPECBRIDGE RETURNS (illustrative)

Affected screens: patient search results, where the button sits, and the start-visit form. The button is built in the patient-chart repo and shown in the patient-management search screen, so two repos are involved.

CASE	STEPS	EXPECTED RESULT
TC-01 happy	Log in as front desk. Search a patient with no active visit. Click **Start visit** on the result.	The start-visit form opens for that patient; after saving, the visit shows as active.
TC-02 error	Same steps, while the server rejects the request.	An "Error starting visit" message appears; no visit is created.

Question for the PM: if the patient already has an active visit, should the button be hidden or open that visit?

The PM reviews this before any code is written. A mismatch, like a different answer to the question above, is caught now instead of at the demo.

5 Build vs rules

Why rules are not enough

- **Free-text input.** Tickets come in any style, from formal criteria to informal Hinglish. Templates enforce structure, not correctness.
- **Language-to-code mapping is semantic.** "Finalize a bill" must be matched to the right code among 2,322 files in 5 repos. On our golden tickets with no file names, keyword search plus code links finds a changed file in the top 10 for 74% of tickets; adding AI search by meaning raises it to 79%. The agent then searches further on its own.
- **Reasoning, not lookup.** Inferring unstated criteria, negative paths and contradictions in a ticket needs judgment.

Where we keep rules (deterministic guardrails)

Team-only ticket intake, secret redaction, read-only AI tools, verification of every cited file and screen, publishing done only by code, cost and time caps.

Accuracy needed and what errors cost

ERROR	COST	TOLERANCE
Leaks a secret, or obeys instructions hidden in a ticket	Security incident	ZERO
Invents a screen or flow that does not exist	Wasted PM/dev time; trust lost	LOW
Misses a case (e.g. an error state)	Bug found later; rework (PM review catches most)	SOME
Awkward wording or a duplicate case	Minor edit	SOME

Output is a draft a human approves, so it need not be perfect; grounding and safety must be near-perfect.

Why AI is worth it

An AI draft costs about $0.02-0.07 and 1-2 minutes per ticket (estimate from our measured token sizes), against an assumed 30-60 minutes of PM time per ticket today, and it brings code knowledge the PM lacks. The time saving is an assumption we will measure in customer sessions.

6 Risks and mitigations

RISK	MITIGATION
Hallucinated screens, steps or code references	Every case must cite code it read; the reviewer verifies each cited file, line and screen name against the code.
Prompt injection or secrets inside tickets	Ticket text treated as data; secrets removed before the AI sees them; AI has read-only tools; only code publishes.
Vague or contradictory tickets	A gate turns them into questions for the PM instead of guessed test cases.
Search misses the right code (79% hit rate on fair tickets today)	Agents search again per criterion; misses are logged and fixed in each improvement round.
The build still drifts from the agreed journey	The approved page is the engineer's direction and the checklist for the demo, so drift is caught step by step.
Overwriting the PM's edits on re-runs	Updates never overwrite human edits; new drafts go in as a page comment.
Cost or latency runaway; free-tier model capacity	Caps of $0.25 and 2 review rounds per ticket; fallback from free to paid capacity.
Our evaluation flatters us	Real tickets fixed after a frozen code version, answer key from the real fixes, links to the fix removed; 18 tickets that name code files scored separately; 6 adversarial tickets.

7 Success metric

Outcome we want: fewer demo-and-rework rounds per UI task.
Baseline: rounds per task today, asked in customer interviews.

Primary (measurable in the capstone): share of generated test cases the PM keeps without edits, i.e. the journey was right before building. Measured in 5+ sessions with real PMs, engineers or QA (not classmates), with verbatims.

Also tracked: mismatches the page surfaced before build (per PM), engineer clarity rating (1-5), tokens, cost and time per ticket.

Test bed: OpenMRS 3, 5 real connected React repos frozen at 31 Mar 2026; golden set of 63 real tickets fixed after that date (answer key from the real code changes) + 6 hand-written adversarial tickets. The team picks and labels the final 30-50.

8 Ship gate (proposed, team to confirm)

MEASURE (GOLDEN SET)	GATE
Acceptance criteria covered by at least one case	≥ 90%
Cited files and screens verified in code (grounding)	≥ 95%
Adversarial tickets handled safely	6 / 6
Human rubric, 2 raters (accuracy, grounding, usefulness)	≥ 4 / 5
LLM judge agrees with humans on spot-checks	≥ 80%
Cost / time per ticket	≤ $0.10 / 3 min

Out of scope: writing or changing code, automated tests, backend design, running the app, auto-approving pages. Sources: team proposal (Oct 2026), project build log and measured search scores (Step 4, 27 Oct 2026).